Laravel Agent Evals — Full Technical Build Specification

Purpose of this document: A complete, self-contained blueprint of the product — what it is, how every piece works, and in what order to build it. Written so that *any* developer or AI assistant, with zero prior context, could read this and build the product correctly.

---

1. What the product actually is (one paragraph)

A Composer package for Laravel that lets a developer define expected AI-agent behavior as test cases, run those test cases against their real agent, automatically compare results to the last known-good run to catch regressions, and optionally block a GitHub pull request when a regression is detected. Distribution is the open-source package itself; monetization comes later as a hosted dashboard and as a done-for-you service.

---

2. The end-to-end flow (what happens, in order, every time it runs)

Developer writes test cases (PHP files)
        │
        ▼
`php artisan agent:eval` is run
        │
        ▼
Runner loads all test cases from tests/AgentEvals/
        │
        ▼
For each test case:
  → send `input` to the configured Agent
  → receive `response` (string)
  → run Assertions (contains / not_contains / callback / judge-score)
  → record EvalResult (pass/fail + reason)
        │
        ▼
Compare full result set to stored Baseline (JSON file)
        │

▼

Any test that PASSED before and FAILS now = Regression

▼

Print report to terminal (or output JSON/HTML for CI/dashboard)

▼

Save current results as the new Baseline (unless --no-baseline-update)

▼

Exit code 0 (all good) or 1 (failure/regression) → this is what makes CI "gate" a PR

This loop is the entire product. Everything else (dashboard, hosted version, judge scoring, capability diffing) is a layer added on top of this loop — never a replacement for it.

---

3. Core components and what each one is responsible for

Component	Responsibility	Status in current MVP
`TestCase`	Describes one expected behavior: an input prompt + assertions	✅ Built
`EvalResult`	Describes the outcome of running one `TestCase`	✅ Built
`EvalRunner`	Orchestrates: load test cases → call agent → assert → compare baseline → save baseline	✅ Built
`AgentEvalCommand` (`agent:eval`)	CLI entry point that runs everything and prints the report	✅ Built
`AgentEvalInitCommand` (`agent:eval:init`)	Scaffolds config + one example test case for first-time setup	✅ Built
`AgentEvalsServiceProvider`	Registers config and commands with the Laravel app	✅ Built
Config file (`agent-evals.php`)	Points the package at the developer's actual agent class/method	✅ Built

Baseline store	JSON file on disk holding the last run's results, used for regression detection	✅ Built (file-based)
GitHub Action	Runs `agent:eval` on every PR and fails the check if the command exits non-zero	✅ Built (example workflow)
Judge scorer	Uses an LLM to fuzzily grade a response instead of exact string matching	⬜ Not built yet — see §6
HTML report	Human-shareable version of the terminal output	⬜ Not built yet — see §6
Hosted dashboard	Web UI showing eval history, trends, team-shared baselines	⬜ Future product layer — see §7
Capability/permission diff	Detects when an agent's tools/permissions changed, not just its answers	⬜ Future product layer — see §7

---

4. Data model (what actually gets stored, in plain terms)

A test case (defined by the developer, lives in PHP files, never stored as data):

```
name:   string, human-readable
input:  string, the prompt sent to the agent
expect: {
  contains?:     string | string[]
  not_contains?: string | string[]
  callback?:     function(response) -> bool
}
tags?:  string[]
```

A baseline entry (one per test case, stored as JSON on disk or, later, in a database):

```
{
  "name": "declines refunds over $100 without approval",
  "passed": true,
  "response": "Refunds over $100 need manager approval...",
  "reason": null
```

}

Regression = a test case whose baseline entry has `passed: true` but whose current run has `passed: false`. That single comparison is the entire "intelligence" of the product at MVP stage — deliberately simple, deliberately reliable.

---

5. Why each technical decision was made (so it doesn't get "improved" into something worse)

- **Composer package, not a SaaS app, at the start.** Zero infrastructure to run, zero signup friction, matches how Laravel developers already install tools (PHPUnit, Pest, Telescope). This is the distribution wedge — don't skip it for a hosted product too early.
- **File-based baseline (JSON on disk), not a database, at the start.** Works with zero setup, works in CI (ephemeral runners), works offline. A database/hosted baseline store is a valid *future* upgrade (§7), not a starting requirement.
- **Simple string assertions before LLM-as-judge.** Deterministic, fast, free, and covers a large share of real test cases (tone/policy/refusal checks). Judge-based scoring adds cost, latency, and non-determinism — only add it once simple assertions are proven insufficient by real usage.
- **CLI-first, dashboard later.** A CLI/CI tool has to work correctly before a UI wrapping it is worth building. The dashboard is a view on top of the same data model in §4, not a separate product.
- **Exit codes drive CI behavior**, not a webhook or API call — this is what lets any CI system (GitHub Actions, GitLab CI, Jenkins) use the package with zero custom integration work.

---

6. Next build layer (in order) — what to build after the current MVP

1. **HTML report output** — add an `--output=html` flag to `agent:eval` that writes a static HTML file (reuse the visual style of the landing page: pass=green, fail=red, monospace). Purely a formatting layer on top of the existing `EvalResult[]` — no new logic required.
2. **LLM-as-judge scorer** — a `judge.class`/`judge.method` config (already stubbed in `config/agent-evals.php`) that sends the response + a rubric to an LLM and returns

a pass/fail + reasoning. Wire it in as an additional assertion type inside `EvalRunner::assert()`.

3. **Multiple baseline branches** — currently one global baseline file; extend to store baselines per git branch so a feature branch doesn't get falsely flagged against `main`'s baseline.
4. **Test case tagging/filtering** — `agent:eval --tags=refunds` to run a subset, useful once a project has hundreds of test cases.
5. **Packagist publishing** — register the package on packagist.org so `composer require ali/laravel-agent-evals` works without a manual path repository. Requires: public GitHub repo, semantic version tag (`v0.1.0`), and a Packagist account linked to the repo (webhook auto-updates on new tags).

---

7. Future product layers (beyond the open-source core — this is where monetization lives)

Layer	What it adds	Depends on
Hosted dashboard	Team-shared eval history, trend charts, Slack/email alerts on regression	A backend API + database that the CLI pushes results to (`agent:eval --report-url=...`)
Managed baselines	Baseline stored centrally instead of per-developer machine, so the whole team shares one source of truth	Same backend as above
Capability/permission diff	Detects when a PR changes an agent's tools, MCP servers, or permission scope — not just its output text	A static analyzer that reads config/tool-registration files in the diff, separate subsystem from the eval runner
Security regression tests	Pre-built adversarial test case packs (prompt injection, data exfiltration attempts) that ship with the package	Content/research work, not new architecture — reuses the exact `TestCase`/`EvalRunner` engine

Certification badge	A generated "Agent Security/Reliability Certificate" once a defined test suite passes, for use in vendor trust/compliance conversations	Builds on the hosted dashboard's stored run history

Important: every layer in this table is additive on top of the same core loop in §2 — none of them require re-architecting the base package. This is intentional: it lets the free open-source core stay simple forever while the paid layers add value around it.

---

8. Folder structure reference (current MVP)

```
laravel-agent-evals/
├── composer.json
├── README.md
├── config/
│   └── agent-evals.php
├── src/
│   ├── AgentEvalsServiceProvider.php
│   ├── TestCase.php
│   ├── EvalResult.php
│   ├── EvalRunner.php
│   └── Console/
│       └── Commands/
│           ├── AgentEvalCommand.php
│           └── AgentEvalInitCommand.php
├── .github/
│   └── workflows/
│       └── agent-evals.yml
└── tests/
    └── (package's own PHPUnit/Pest tests go here — not to be confused
        with the end-user's AgentEvals test cases, which live in
        *their* Laravel app at tests/AgentEvals/)
```

---

9. Glossary (for handoff to anyone/any AI with zero context)

- **Test case** — one defined expectation of agent behavior (input + what the response must/must not contain).

- **Baseline** — the stored result of the last run, used as the comparison point for regression detection.
- **Regression** — a test that used to pass and now fails.
- **Runner** — the engine that executes test cases against the agent and produces results.
- **Judge / LLM-as-judge** — using a second LLM call to grade a response instead of simple string matching, for cases too fuzzy for `contains`/`not_contains`.
- **Gate** — the CI behavior of blocking a merge when the eval run's exit code is non-zero.
- **Capability diff** (future layer) — detecting changes to what tools/permissions an agent has access to, independent of what it actually says in a response.

---

10. One-sentence instruction for handing this to another AI

"Build a Laravel Composer package that lets a developer define AI-agent test cases in PHP, run them via an Artisan command, compare each run against a stored JSON baseline to detect regressions, exit non-zero on failure so it can gate a GitHub Actions PR check — starting exactly from the architecture and file structure in sections 3, 4, and 8 of this document, then layer in §6 and §7 in order."